

Deploy Spark Cluster on Google Cloud Dataproc:

Cloud Dataproc is a fast, easy-to-use, fully-managed cloud service for running Apache Spark and Apache Hadoop clusters in a straightforward and cost-efficient way. Google's documentation now calls this service Managed Service for Apache Spark; the gcloud commands below still use "dataproc".

Create a bucket (if you haven't already done so) – The following steps need to be taken **only once**:

1. Log in to your Google Cloud account (<https://console.cloud.google.com>) and go to Cloud Storage.
2. Create a new bucket. Call it qst843-xx (xx your initial)
3. Select **multi-region**, and for location, select **us**, leave the rest as default, and click **Create**.
4. Run the following query to test if everything is working fine from Cloud Shell.
 - a. `gcloud storage ls`
 - b. This should list all the buckets in your project, including the one you just made.

Create a cluster –Open a Google Cloud Shell. Make sure you are in the intended project.

1. **New project?** Turn on the two APIs this guide uses (once per project). If a gcloud command later asks to enable an API, answer y.

```
gcloud services enable compute.googleapis.com dataproc.googleapis.com
```

2. For added security, clusters are set up without an external IP address, which results in a lack of internet access. We can use Cloud NAT service to allow instances that don't have external IP addresses to connect to the internet securely. Create Cloud Router instances in *us-central1* region. Enable Private Google Access on the default Subnetwork and internet access to the VM. **Add Cloud NAT** for your VPC/subnet/region (default/default/us-central1):

```
gcloud compute networks subnets update default \
--region us-central1 \
--enable-private-ip-google-access

gcloud compute routers create nat-router-us-central1 \
--network="default" --region="us-central1"

gcloud compute routers nats create nat-config \
--router=nat-router-us-central1 \
--region="us-central1" \
--nat-custom-subnet-ip-ranges="default" \
--auto-allocate-nat-external-ips
```

We should now have internet access on the master node, which will enable us to perform tasks such as software updates or git clones. This step needs to be done only once per project/region.

Why this matters: Without Cloud NAT the cluster still starts, but it cannot reach the internet (no pip install or git clone from the cluster). Without Private Google Access, the very first cluster in a new project can fail with "The resource ... subnetworks/default is not ready"; if that happens, simply run the create command again.

Give the cluster permission to use your bucket (once per project). The cluster VMs run as your project's default Compute Engine service account. In newer projects that account has no roles, and cluster creation fails with "Permissions are missing for the default service account ...

storage.buckets.get, storage.objects.create ...". Run the following in Cloud Shell to grant it the Dataproc Worker role, then wait about a minute before creating the cluster:

```
PROJECT_ID=$(gcloud config get-value project)
PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --
format="value(projectNumber)")
gcloud projects add-iam-policy-binding $PROJECT_ID \
--member="serviceAccount:${PROJECT_NUMBER}-
compute@developer.gserviceaccount.com" \
--role="roles/dataproc.worker"
```

If this command fails with a permission error, your account cannot change IAM in this project; ask the project owner to run it.

3. The following step must be performed **to create a new cluster**. Run the following script in Cloud Shell to launch a new cluster named *single-spark353*. Copy this command to an editor of your choice and make sure to replace <BUCKET-NAME> with the bucket you created in the previous section (3-5 minute):

```
gcloud dataproc clusters create single-spark353 \
--bucket <BUCKET-NAME> \
--region us-central1 \
--master-machine-type n2-highmem-4 \
--master-boot-disk-type pd-balanced \
--master-boot-disk-size 30 \
--single-node \
--no-address \
--image-version 2.3-debian12 \
--optional-components JUPYTER \
--enable-component-gateway \
--stop-max-idle 30m
```

(This script is also accessible in 06-Hands-on-Spark/Deploy-Dataproc-Cluster-Single-Node.txt)

- This will give you a single-node cluster with a 30 GB disk, 4 vCPUs, and 32 GB memory (n2-highmem-4), running Spark 3.5. It costs about \$0.30/hour while running (VM plus the Dataproc fee), and it stops itself after 30 idle minutes (--stop-max-idle 30m). The google-cloud-storage Python package is already installed on the image, so no initialization action is needed.
4. Confirm that your cluster is up and running from the Dataproc page (type Dataproc in the search bar at the top of the console; Google now titles the page Managed Service for Apache Spark).
 5. Copy the course materials into your bucket (only once). In Cloud Shell, run:

```
git clone --depth 1 https://github.com/soltaniehha/Big-Data-Analytics-for-Business.git
gcloud storage cp -r Big-Data-Analytics-for-Business gs://<BUCKET-
NAME>/notebooks/jupyter/
```

The repository then shows up in JupyterLab under the GCS folder, and the notebooks read their data from gs://<BUCKET-NAME>/notebooks/jupyter/Big-Data-Analytics-for-Business/data/.

Connecting to the Cluster

- From the Dataproc tab, select the cluster, and under web interfaces, select JupyterLab.

- OR - From the Cloud SDK shell, execute the following command to establish a secure SSH tunnel:

```
gcloud compute ssh --zone us-central1-X single-spark353-m --tunnel-through-iap -- -N -L 8123:localhost:8123 -L 8088:localhost:8088
```

Replace X with the cluster's zone (the last part of the output of: `gcloud dataproc clusters describe single-spark353 --region us-central1 --format="value(config.gceClusterConfig.zoneUri)"`). Then open `http://localhost:8123/gateway/default/jupyter/lab` in your browser (plain `localhost:8123` shows a 404 page). The YARN UI is at `http://localhost:8088`.

Note: You may occasionally get disconnected. If that happens, repeat the step above and refresh your browser. Your work will be saved in your cluster and bucket under the *notebook* folder.

Stopping your Cluster

To ensure you don't run out of money, stop your instance once you are not using it. You can do this by going to the Dataproc page and stopping the cluster. From Cloud Shell:

```
gcloud dataproc clusters stop single-spark353 --region us-central1
```

(resume later with `gcloud dataproc clusters start single-spark353 --region us-central1`). A stopped cluster only costs its disk (about 10 cents a day for the 30 GB disk).

Clean up

To make sure we won't be charged for any of the resources we are not using, delete the cluster after you are done with the course:

- Select the cluster from the Dataproc page and click the DELETE button.
 - Alternatively, you can also use the following command from a **new** Cloud SDK terminal (please check from the UI to make sure the cluster is being deleted):

```
gcloud dataproc clusters delete single-spark353 --region us-central1
```

* Notice that even after deleting the cluster, your notebooks will persist in the bucket, and when you create a new cluster that points to the same bucket, you can reuse those notebooks.

Deploying a Large Cluster

To create a large cluster (**CAUTION**) with multiple nodes, use the following command:

```
gcloud dataproc clusters create multi-spark353 \  
  --bucket <BUCKET-NAME> \  
  --region us-central1 \  
  --master-machine-type c3-highmem-4 \  
  --master-boot-disk-type pd-balanced --master-boot-disk-size 30 \  
  --num-workers 4 \  
  --worker-machine-type c3-highmem-4 \  
  --worker-boot-disk-type pd-balanced --worker-boot-disk-size 30 \  
  --image-version 2.3-debian12 \  
  --no-address \  
  --optional-components JUPYTER \  
  --enable-component-gateway \  
  --stop-max-idle 2h
```

(This script is also accessible in 06-Hands-on-Spark/Deploy-Dataproc-Cluster-Multi-Node.txt)

Important note: Please be advised of its cost. Cost can be calculated using Dataproc cost calculator (make sure to include GCE is selected): <https://cloud.google.com/products/calculator/>

The cluster above has 1 master node, 4 primary workers, and 0 secondary workers (5 × c3-highmem-4: 16 vCPUs and 128 GB memory on the workers). It needs 20 C3 vCPUs of quota in us-central1 (IAM & Admin > Quotas). Spot secondary workers are much cheaper (60–90% off), but Google can reclaim them at any time, and a cluster with secondary workers cannot be stopped, only deleted, so it's not ideal for our use-case. To add them anyway, append these flags:

```
--secondary-worker-type spot --num-secondary-workers 2 --secondary-worker-boot-disk-size 30
```

This cluster will cost about \$1.55/hour (VMs plus the Dataproc fee).

Note about the Spot worker pool: One can easily add/remove Spot workers from the UI (or in the command line). Go to the cluster of interest in Dataproc (make sure the cluster is running) > Configuration > Edit > add/remove secondary workers > save. After ~30 seconds your cluster will reflect the change.

Connecting to the Cluster

Same as the single node cluster mentioned above.

Clean up

```
gcloud dataproc clusters delete multi-spark353 --region us-central1
```